

Creación de Procesos en POSIX

Malak Sanchez

Monitorias Sistemas Operativos

1 de septiembre de 2026

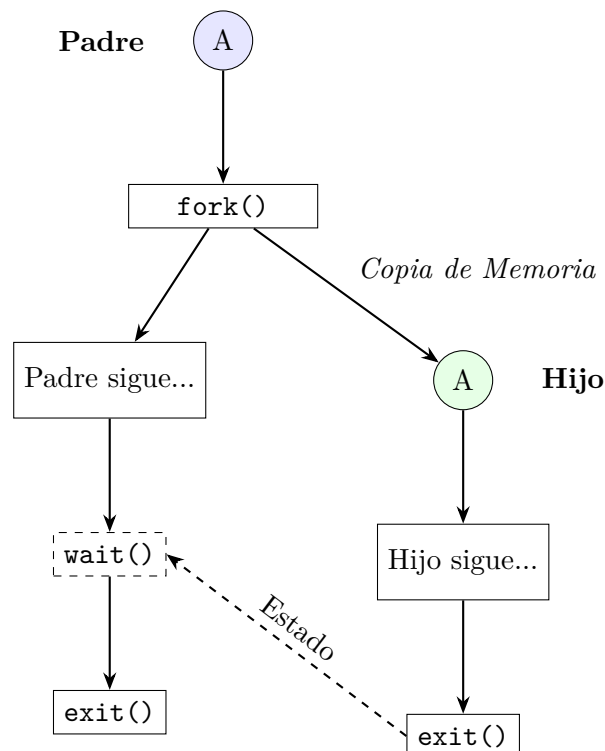
1. Motivación

En un sistema operativo moderno, los programas no siempre corren de forma aislada. A menudo, un programa necesita delegar tareas a otro proceso para realizarlas en paralelo o de forma independiente. En Linux/POSIX, la forma fundamental de crear un nuevo flujo de ejecución es “duplicando” el proceso actual.

2. Idea Fundamental

La función `fork()` crea un nuevo proceso (Hijo) que es una **copia exacta** del proceso que lo llamó (Padre). Ambos procesos continúan ejecutándose desde el mismo punto exacto (el retorno de `fork()`).

3. Arquitectura de Ejecución



4. Sintaxis Básica

```
#include <unistd.h>
#include <sys/types.h>

pid_t pid = fork();
```

5. Ejemplo Mínimo Funcional

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main() {
5     pid_t pid = fork();
6
7     if(pid < 0){
8         perror("fork failed");
9         return 1;
10    }else if(pid == 0){
11        printf("I am the child process!\n");
12    }else{
13        printf("I am the parent! My child's PID is %d\n", pid);
14    }
15
16    return 0;
17 }
```

6. Explicación Paso a Paso

1. Se llama a `fork()`: el Sistema Operativo crea una copia del proceso.
2. **Valor de retorno:** Es la clave para diferenciar quién es quién.
 - Al **Padre** se le devuelve el PID del hijo (un número > 0).
 - Al **Hijo** se le devuelve el valor **0**.
 - Si hay error, devuelve -1 .
3. A partir de la línea 6, ambos procesos corren el mismo código pero con valores de `pid` diferentes.

7. Qué ocurre en memoria (COW)

Linux utiliza una técnica llamada **Copy-On-Write (COW)**. El proceso hijo comparte la misma memoria física que el padre hasta que alguno intenta **escribir** en ella. En ese instante, el Sistema Operativo duplica la página de memoria para que cada uno tenga su propia versión privada de los datos.

8. Patrones comunes de uso

El patrón `switch` es muy común cuando se tienen múltiples casos de error, hijo y padre:

```
1 switch(fork()) {  
2     case -1: perror("error"); exit(1);  
3     case 0:  /* Código del hijo */ break;  
4     default: /* Código del padre */ break;  
5 }
```

9. Errores Comunes

- **Bomba lógica:** Hacer un `fork()` dentro de un bucle infinito sin control puede agotar los recursos del sistema (Fork Bomb).
- **Asumir orden:** El padre y el hijo corren de forma asíncrona. No puedes saber cuál imprimirá primero sin usar sincronización.

10. Buenas Prácticas

- Siempre validar el retorno de `fork()`.
- Usar el tipo de dato `pid_t` (definido en `sys/types.h`) para manejar Identificadores de Proceso.

11. Ejercicios

1. Escriba un programa donde el hijo imprima las letras de A a E y el padre las letras de F a J.
2. Declare un arreglo global antes de `fork()`; haga que el hijo modifique uno de sus elementos y compare el arreglo que imprime cada proceso. Explique el resultado.